

B.Sc. (Hons) in Software Development



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

AI Phishing Email Detection with Explainable AI

By
NATHAN CARR

April 26, 2026

Minor Dissertation

**Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.**

Contents

1	Introduction	1
1.1	Understanding Phishing Attacks	1
1.2	Why AI is Needed for Phishing Detection	1
1.3	Problem Statement	2
1.4	Research Aim and Objectives	2
1.5	Scope and Limitations	2
1.6	Overview of the Dissertation	3
2	Research and Development Methodology	4
2.1	Overall Development Approach	4
2.2	Planning and Requirements	4
2.3	Technology and Tool Selection	4
2.4	Empirical Approach and Problem Solving	5
2.5	Validation and Testing	5
2.6	Dataset Construction and Preprocessing	5
2.7	Feature Extraction and Model Selection	6
3	Technology Review	7
3.1	Common Phishing Email Techniques	7
3.2	Machine Learning for Email Classification	7
3.3	Text Feature Extraction	8
3.4	Evaluation Metrics	8
3.5	Explainable Artificial Intelligence	8
3.5.1	LIME	8
3.5.2	SHAP	8
3.6	The MITRE ATT&CK Framework	9
3.7	Research Gaps and Project Justification	9
4	System Design and Architecture	10
4.1	System Overview	10
4.2	Data Processing Pipeline	11
4.3	Model Training and Explainability	11
4.4	Attacker Simulation Design	12
4.5	MITRE ATT&CK Mapping	12
4.6	Reproducibility and CI	12

5	Implementation	13
5.1	Overview	13
5.2	Dataset Builder	13
5.3	Baseline Classifier	13
5.4	Explainability Integration	13
5.5	Rigorous Evaluation and Error Analysis	14
5.6	Attacker Simulation	14
5.7	MITRE ATT&CK Mapping	15
5.8	Streamlit Dashboard	15
5.9	Testing	15
6	Experimental Results and Evaluation	16
6.1	Baseline Model Performance	16
6.1.1	Classification Results	16
6.1.2	Confusion Matrix and Error Analysis	16
6.1.3	ROC Curve Comparison	17
6.2	Cross-Validation and Rigorous Evaluation	17
6.2.1	Cross-Validation Stability	17
6.2.2	Threshold Sensitivity and Calibration	18
6.3	Explainability Results	18
6.3.1	LIME Explanations	18
6.3.2	SHAP Global Feature Importance	19
6.3.3	Streamlit Dashboard	20
6.4	Attacker Simulation Results	21
6.4.1	Effect of Classification Threshold	21
6.4.2	Detection Rate by Attack Type and Rule Chain	21
6.4.3	Detection Rate by MITRE Technique	22
6.5	MITRE ATT&CK Mapping Evaluation	23
7	Discussion	24
7.1	Trust and Transparency in AI-Based Phishing Detection	24
7.2	Strengths of the Proposed System	24
7.3	Limitations of the Current Approach	25
7.4	Implications for Real-World Deployment	25
8	Conclusion and Future Work	27
8.1	Summary of the Project	27
8.2	Achievement of Objectives	27
8.3	Limitations	28
8.4	Recommendations for Future Work	28
8.4.1	Comparison with Deep Learning Models	28
8.4.2	Adversarial Attack Testing	28
8.4.3	URL and Metadata Features	28
8.4.4	REST API Deployment	28
8.4.5	Dataset Modernisation	28
A	Code Excerpts and Repository	29
B	Additional Evaluation Results	35

C Simulation Logs	39
D MITRE ATT&CK Mapping Tables	42

List of Figures

6.1	ROC curve: Logistic Regression (AUC = 0.9808).	17
6.2	ROC curve: Naive Bayes (AUC = 0.9642).	17
6.3	Top 20 global SHAP feature importance scores computed across 2000 training emails.	19
6.4	The Streamlit dashboard showing the input screen with an example phishing email loaded.	20
6.5	Analysis results showing 83.4% phishing confidence, T1566.002 MITRE mapping, and LIME explanation for the account suspension email.	20
6.6	Detection rate by threshold.	21
6.7	Bypass rate by threshold.	21
6.8	Detection rate by attack type.	21
6.9	Detection rate by rule chain.	22
6.10	Detection rate by MITRE ATT&CK technique.	22

List of Tables

4.1	Source modules and their responsibilities.	11
4.2	Dataset sources and their assigned labels.	11
4.3	Attack rules used in the simulation.	12
6.1	Classification report for the Logistic Regression baseline model on the held-out test set.	16
6.2	Confusion matrix for the baseline model.	16
6.3	Cross-validation summary (5 folds $\times$ 3 repeats = 15 runs).	18
6.4	Threshold sensitivity for Logistic Regression. Best F1 at threshold 0.40. . .	18
6.5	MITRE mapping evaluation against a manually labelled validation subset (15 samples).	23
A.1	Source files in the <code>src/</code> directory.	30
A.2	Test files in the <code>tests/</code> directory.	30
B.1	Full cross-validation results (5 folds $\times$ 3 repeats = 15 runs). CI bounds are 2.5th and 97.5th percentiles across fold scores.	35
B.2	Single-split test set comparison. Logistic Regression outperforms Naive Bayes across all five metrics.	35
B.3	Full threshold sensitivity for Logistic Regression across all 18 thresholds. Optimal F1 is at threshold 0.40, highlighted in bold.	36
B.4	Simulation results by threshold. Precision is 1.000 at all thresholds — no legitimate emails were ever misclassified as phishing.	36
B.5	Full simulation results by rule chain, sorted by detection rate. Results are aggregated across all thresholds and simulation rounds.	37
B.6	Error analysis summary from the test set.	37
B.7	Top distinctive tokens in false positive and false negative emails. FP tokens include financial vocabulary the model associates with phishing. FN tokens are common function words with no distinctive phishing signal.	38
C.1	Column descriptions for the attacker simulation log CSV.	39
C.2	Representative simulation log excerpt at threshold 0.5. The <code>spoof_revenue</code> probability of 0.447 falls below the detection threshold. Any chain including <code>fake_link</code> produces probabilities at or near 1.0.	40
C.3	LIME feature weights for the <code>spoof_revenue</code> bypass email. Negative weights indicate words pushing the prediction toward legitimate. <code>eligible</code> and <code>Revenue</code> both reduce the phishing probability, explaining why the model missed this attack.	40

C.4	Cross-tabulation of threshold and attack type. At threshold 0.5, every multi-rule attack is detected. Single-rule attacks are harder at every threshold.	40
C.5	Simulation results by MITRE technique. The generic fallback category has a bypass rate of 88.9%, confirming that emails without detectable patterns are almost invisible to the model.	41
D.1	MITRE ATT&CK techniques with associated patterns and keywords. . . .	42
D.2	Full MITRE mapping evaluation results against a 15-sample manually labelled validation subset.	43
D.3	Per-label precision, recall, and F1 for the primary-label MITRE mapping evaluation. Three of five techniques achieved perfect scores. T1566.001 occasionally over-assigns (precision 0.75). T1598.001 is the hardest to identify (recall 0.667).	43
D.4	Simulation detection rates by MITRE technique. The generic fallback category, assigned when no patterns match, has the lowest detection rate at 0.111 and the highest bypass rate at 0.889.	43
D.5	Attack rule to MITRE technique mapping with single-rule detection rates. spoof_revenue maps to the generic fallback category because it contains no URL patterns or attachment keywords, producing the lowest detection rate in the simulation.	44

Chapter 1

Introduction

1.1 Understanding Phishing Attacks

Phishing is one of the most common and consistently effective forms of cyberattack. An attacker sends an email designed to look like it came from a trusted source, with the goal of tricking the recipient into clicking a malicious link, handing over credentials, or transferring money. What makes phishing particularly difficult to deal with is not the technical sophistication of the attack but the social engineering behind it. Urgency, impersonation, and fear are used to bypass rational thinking and get people to act before they have time to question whether the email is genuine.

Phishing remains one of the primary entry points for data breaches and ransomware attacks, and it continues to evolve. Modern phishing emails use fluent language, realistic branding, and contextual details that earlier attacks simply did not have. More recently, attackers have started using large language models to generate phishing content at scale, producing emails that are grammatically correct and contextually convincing in a way that traditional detection systems were not built to handle [1, 2].

1.2 Why AI is Needed for Phishing Detection

Rule-based email filters check messages against known patterns such as blacklisted domains or specific phrases. The problem is that these rules are static. Once attackers figure out what the filter is looking for, they change their approach. This cat and mouse dynamic means rule-based systems require constant manual updating and will always lag behind the latest attack techniques.

Machine learning offers a different approach. Instead of following fixed rules, a model learns patterns directly from data. Given enough examples of phishing and legitimate emails, it can develop its own understanding of what distinguishes them, making it more adaptable to new attack styles.

The challenge is that most machine learning models are black boxes. They produce a prediction without any explanation of why. In a security context this is a real problem. If a system flags an email as phishing and a security analyst cannot see any reason why, it is hard to know whether to trust the decision. It also makes it very difficult to identify cases where the model is getting things wrong. This is where explainable AI comes in, and it is the central motivation for my project.

1.3 Problem Statement

Despite strong detection performance on benchmark datasets, most machine learning approaches to phishing detection are not transparent about how they reach their decisions. Recent research shows that only 39% of phishing detection studies implement any form of explainability [3], even though the lack of transparency is widely acknowledged as a limitation. There is also a tendency in the literature to evaluate models only on standard train/test splits, which do not reflect the reality that real attackers actively try to evade detection systems.

This project addresses both of these gaps by building a phishing detection system that explains its predictions and evaluating it through an attacker simulation, repeated cross-validation, and a formal assessment of the MITRE ATT&CK mapping.

1.4 Research Aim and Objectives

The aim of my project is to develop a phishing email detection system that combines machine learning with Explainable AI, and to evaluate that system in a way that goes beyond standard classification metrics.

The specific objectives were:

- To build and train a phishing detection model using TF-IDF and Logistic Regression on a balanced dataset of 21,402 real emails, with Multinomial Naive Bayes trained alongside for comparison
- To evaluate the model using standard metrics on a held out test set, supported by repeated stratified cross-validation (5 folds, 3 repeats) to produce confidence intervals and confirm result stability
- To integrate LIME and SHAP explainability so that individual predictions and global feature importance can be understood, with a linear weight fallback when these are unavailable
- To develop an attacker simulation module testing the model against modified phishing emails across four attack rules, twelve two-rule combinations, and three classification thresholds
- To map detection results to the MITRE ATT&CK framework and formally evaluate mapping accuracy against a manually labelled validation subset
- To build an interactive Streamlit dashboard allowing email text or .eml files to be submitted for live classification, explanation, and MITRE technique mapping
- To implement a dedicated error analysis module extracting and analysing false positive and false negative cases from the test set

1.5 Scope and Limitations

This project focuses on detecting phishing emails based on text content, trained on publicly available English language datasets.

The attacker simulation uses four predefined rules applied individually and in pairs. While more realistic than a static test set, it does not fully replicate real attacker behaviour, which is adaptive and not constrained to fixed rules.

The datasets used are: Enron for legitimate emails, and SpamAssassin, Nazario, and Nigerian Fraud for phishing, are publicly available but dated. The Enron corpus dates from 1999 to 2002, and 72% of recent phishing detection studies similarly rely on datasets more than eight years old [3]. The SHAP analysis in this project confirms this concern directly: several of the highest scoring global features are Enron specific terms rather than genuine phishing vocabulary. The results should be understood as a proof of concept rather than a production-ready system.

1.6 Overview of the Dissertation

Chapter 2 covers the methodology. Chapter 3 reviews the relevant literature. Chapter 4 describes the system design and architecture. Chapter 5 covers the implementation in detail including code snippets and the test suite. Chapter 6 presents the experimental results. Chapter 7 discusses the findings and their implications. Chapter 8 concludes and recommends future work.

The project repository is available at:

<https://github.com/NathanCarr10/fyp-phishing-detection-mitre-xai>

A screencast demonstration of the system is available at:

https://youtube.com/watch?v=CVL3C_xHcic

Installation instructions and a description of repository contents are in Appendix A.

Chapter 2

Research and Development Methodology

2.1 Overall Development Approach

This project combined a research methodology with an iterative software development approach. The research side involved reviewing existing literature on phishing detection, explainable AI, and the MITRE ATT&CK framework before any code was written, ensuring that technical decisions had solid reasoning behind them. The development side used an Agile approach with two-week sprints, where each component was built, tested, and reviewed before the next was started. Progress was reviewed regularly with my supervisor to assess whether the current stage was working and what the next steps should be.

2.2 Planning and Requirements

The main components identified at the start were:

- Dataset construction, balancing, and preprocessing
- Baseline model training and evaluation (Logistic Regression and Naive Bayes)
- Explainability integration (LIME, SHAP, and linear weight fallback)
- MITRE ATT&CK mapping and formal mapping evaluation
- Attacker simulation module
- Rigorous model evaluation with cross-validation and calibration analysis
- Error analysis module
- Interactive Streamlit dashboard

2.3 Technology and Tool Selection

Python was chosen as the main programming language due to its ecosystem for machine learning and data processing. `scikit-learn` provided implementations of TF-IDF, Logistic Regression, and Naive Bayes. LIME and SHAP were selected after reviewing the

available explainability options — LIME for local per-email explanations and SHAP for global feature importance. A linear weight fallback was also implemented using Logistic Regression coefficients for when neither library is available. Streamlit was chosen for the interface as it allows a functional web application to be built entirely in Python.

Git and GitHub were used for version control, with a GitHub Actions CI workflow (`.github/workflows/ci.yml`) configured to automatically run the test suite on every push. This meant any change breaking existing functionality would be flagged immediately.

2.4 Empirical Approach and Problem Solving

An empirical approach was taken throughout development. Rather than committing to a single design upfront, options were tested and evaluated based on measurable outcomes. The 80/20 train/test split is a standard and well-justified choice for datasets of this size. The dataset was balanced by downsampling the majority class before splitting, and performance was measured using accuracy, precision, recall, F1-score, and ROC-AUC to give a rounded view rather than relying on a single metric. Multinomial Naive Bayes was trained alongside Logistic Regression to provide a comparison point.

Problems were investigated step by step. Differences in column naming across source datasets were resolved by building automatic column detection into the dataset construction script rather than hardcoding fixes per source. To support reproducibility, `src/reproduce_pipeline.py` runs all pipeline stages in sequence from a single command, controlling the random seed and saving a JSON metadata file recording the Python version, platform, and runtime of each stage.

2.5 Validation and Testing

Validation was carried out at multiple levels. Each module was tested individually before integration. At the model level, a stratified 80/20 train/test split was used as the primary evaluation, with repeated stratified cross-validation (5 folds, 3 repeats, 15 runs per model) run to confirm result stability and produce 95% confidence intervals. The cross-validation also included threshold sensitivity analysis from 0.10 to 0.95, and calibration analysis using Brier score and Expected Calibration Error. The MITRE mapping was formally evaluated against a manually labelled validation subset. Explainability outputs were manually reviewed for a sample of predictions to confirm they highlighted sensible features. Unit tests were written using `pytest` covering utility functions, the XAI explainer, email ingestion, and evaluation helpers. The test suite is described in Chapter 5 and run with:

```
python -m pytest tests/ -v
```

2.6 Dataset Construction and Preprocessing

The dataset was assembled by combining four publicly available email sources. Legitimate emails came from the Enron corpus and phishing examples from SpamAssassin, Nazario, and Nigerian Fraud collections. Using multiple sources reduces the risk of the model overfitting to the patterns of a single dataset. The combined and balanced dataset contains

21,402 emails with equal representation of both classes. All datasets are publicly available and were used solely for academic purposes.

A custom script (`src/build_dataset.py`) handles differences in column naming by automatically detecting the text column, removes empty entries, shuffles the data to prevent ordering bias, and downsamples the majority class to produce the balanced version. Preprocessing was kept minimal beyond string normalisation, as over-cleaning risked stripping out the urgency cues and suspicious phrases the model needs to learn from.

2.7 Feature Extraction and Model Selection

TF-IDF was used to convert email text into numerical features, configured with 5,000 maximum features, lowercase normalisation, and English stopword removal. It was chosen for its simplicity and direct compatibility with LIME and SHAP, because the features correspond to individual words, the explanations are easy to interpret.

Logistic Regression was selected as the primary classifier for its interpretability and natural integration with the explainability tools. `class_weight="balanced"` was used to handle remaining class imbalance. More complex models such as neural networks were considered but set aside because the added complexity would have made explainability outputs much harder to interpret, which would have worked against the core goal of the project.

Chapter 3

Technology Review

This chapter reviews the key technologies and frameworks that informed this project, covering phishing techniques, machine learning for email classification, feature extraction, evaluation metrics, explainable AI, and the MITRE ATT&CK framework.

3.1 Common Phishing Email Techniques

Phishing emails remain one of the most effective cyberattack methods. Unlike earlier campaigns that relied on obvious spelling errors or generic messages, modern phishing emails use fluent language, realistic branding, and contextual awareness [2, 1]. The most common techniques include urgency cues, impersonation of trusted organisations, and malicious links disguised as legitimate URLs [4]. Attackers are increasingly using large language models to generate phishing content at scale, raising serious questions about whether existing detection systems can keep pace [1, 5].

3.2 Machine Learning for Email Classification

Machine learning has become the dominant approach for phishing detection, replacing rule-based filters that struggled to adapt to evolving attack strategies [6]. Early approaches such as Naive Bayes, SVMs, and Random Forests showed reasonable performance but relied on manually engineered features, limiting their ability to generalise [4]. Deep learning models such as CNNs and LSTMs improved on this by learning representations directly from raw text [7, 8].

Transformer-based models such as BERT and RoBERTa are now the state of the art, achieving detection accuracies of 94–98% on benchmark datasets [9, 10]. However, they may be less robust than benchmark results suggest. Wang et al. [2] found that a BERT-based model dropped from 96.7% to 89.3% accuracy when tested against LLM-generated phishing, a 7.4 percentage point decline that suggests benchmark performance does not always carry over to modern threats.

For this project, Logistic Regression was chosen over a transformer, prioritising interpretability over raw performance. A Multinomial Naive Bayes model was trained alongside it as a comparison point.

3.3 Text Feature Extraction

TF-IDF weights words based on how frequently they appear in a specific email relative to the whole dataset, giving higher weight to vocabulary that is distinctive to individual emails. It is widely used with traditional classifiers and I believe was the right choice here because its features correspond directly to individual words, making them straightforward to highlight in LIME and SHAP explanations [4, 11]. In my opinion, more complex approaches such as word embeddings or contextual transformer representations would have made explainability outputs significantly harder to interpret.

3.4 Evaluation Metrics

In a security context the cost of different errors is not equal, so a range of metrics is needed rather than accuracy alone.

Precision measures the proportion of phishing predictions that were correct. **Recall** measures the proportion of actual phishing emails that were detected, arguably the more important metric in a security setting, since a missed phishing email means an attack got through. **F1-Score** combines both as their harmonic mean and is the most commonly reported metric in phishing detection research, appearing in 83% of studies [3]. **ROC-AUC** measures separability across all possible thresholds, giving a broader picture of classifier performance, though it appeared in only 33% of recent studies [3]. The **confusion matrix** breaks results into true and false positives and negatives, making the type and consequences of errors visible.

This project also uses repeated stratified cross-validation for confidence intervals, threshold sensitivity analysis to find the optimal operating point, and Brier score and Expected Calibration Error (ECE) to assess how well probability outputs reflect actual phishing likelihood. All metrics are discussed in Chapter 6.

3.5 Explainable Artificial Intelligence

Most modern phishing detection models give no indication of why they made a decision. For a security analyst verifying a flagged email this is a real problem, and it makes it harder to catch cases where the model is wrong for the wrong reasons [12, 13]. Despite this, only 39% of phishing detection studies implement any explainability [3].

3.5.1 LIME

LIME works by making small changes to the input text and observing how the prediction shifts, then fitting a simpler interpretable model around that specific prediction to approximate the original model's local behaviour [13]. For phishing detection, this highlights the words that pushed the classification toward phishing or legitimate. The main limitation is inconsistency: because LIME relies on random perturbations, running it twice on the same email can produce different results [13, 14].

3.5.2 SHAP

SHAP assigns each feature a contribution score based on how much it influenced the prediction across all possible feature combinations, making it more theoretically consistent

than LIME [13]. It also works globally, showing which words were most important across the whole dataset rather than just for one email at a time [12]. In my project, SHAP is implemented using `shap.LinearExplainer`, which is optimised for linear models and uses a small background sample to compute contributions efficiently, making it fast enough for real-time use in the Streamlit interface. Linear weight analysis is retained as a fallback when neither LIME nor SHAP is available.

3.6 The MITRE ATT&CK Framework

MITRE ATT&CK is a publicly available knowledge base documenting the tactics, techniques, and procedures used by real-world attackers. Several techniques are directly relevant to phishing: T1566.002 (Phishing: Link) and T1566.001 (Phishing: Attachment) cover the most common email-based initial access methods, with T1598.001, T1598.002, and T1598.003 covering more targeted spearphishing variants. Adding MITRE mappings to detection outputs gives analysts a concrete attack category rather than just a probability score, connecting the system to a framework that security teams already use. In my project the mapping is formally evaluated against a manually labelled validation subset rather than just used as a labelling tool.

3.7 Research Gaps and Project Justification

The literature points to three gaps this project addresses directly.

First, transformer-based models dominate recent research but rarely include explainability. High accuracy is not enough if a security analyst cannot understand why an email was flagged [3].

Second, 72% of studies rely on datasets more than eight years old [3]. The SHAP analysis in this project provides direct evidence of the resulting problem: top-ranked global features include Enron-specific terms like `enron`, `ect`, and `spamassassin` rather than genuine phishing vocabulary — a concrete illustration of what outdated training data looks like in practice.

Third, very few studies evaluate how reliable their explanations actually are [13, 12]. Generating an explanation is not the same as generating an accurate one.

This project addresses all three by building LIME, SHAP, and linear weight explanations into the core of the system, connecting outputs to MITRE ATT&CK, and choosing Logistic Regression specifically because interpretability was the priority over chasing benchmark accuracy [12, 13].

Chapter 4

System Design and Architecture

4.1 System Overview

The system is made up of fifteen Python modules that each handle a different part of the process and work together as a pipeline. The process starts with dataset preparation, where four raw email datasets are merged, cleaned, and balanced. That dataset trains two models using TF-IDF feature extraction: Logistic Regression as the primary classifier and Multinomial Naive Bayes for comparison. After training, the models are saved to disk and loaded by the other components as needed. The explainability module provides email explanations using SHAP, LIME, or linear weights as a fallback. The attacker simulation tests the model against modified phishing emails and maps results to MITRE ATT&CK. A rigorous evaluation module runs cross-validation and calibration analysis. A dedicated error analysis module examines false positives and false negatives. Finally, a Streamlit dashboard ties everything together for interactive use.

Table 4.1 lists all fifteen source modules and their responsibilities.

Module	Responsibility
<code>build_dataset.py</code>	Dataset construction and balancing
<code>mvp_baseline.py</code>	Model training, evaluation, LIME, SHAP
<code>models.py</code>	Reusable model factory and evaluation helpers
<code>xai_explainer.py</code>	SHAP, LIME, and linear weight explanations
<code>attacker_sim.py</code>	Attacker simulation and MITRE mapping
<code>analyse_simulation_results.py</code>	Simulation log processing
<code>visualise_results.py</code>	Chart generation
<code>compare_models.py</code>	Model comparison
<code>evaluate_models_rigorously.py</code>	Cross-validation, threshold sensitivity, calibration
<code>evaluate_mitre_mapping.py</code>	Formal MITRE mapping evaluation
<code>run_error_analysis.py</code>	False positive and negative analysis
<code>reproduce_pipeline.py</code>	Pipeline runner with seed control
<code>email_ingestion.py</code>	.eml file parsing
<code>utils.py</code>	Shared constants and helpers
<code>app.py</code>	Streamlit dashboard

Table 4.1: Source modules and their responsibilities.

4.2 Data Processing Pipeline

`src/build_dataset.py` loads four CSV files from the `data/raw` directory, each representing a different email source.

Dataset	Label
Enron.csv	0 (Legitimate)
SpamAssassin.csv	1 (Phishing)
Nazario.csv	1 (Phishing)
Nigerian_Fraud.csv	1 (Phishing)

Table 4.2: Dataset sources and their assigned labels.

A `pick_text_column` function automatically detects the correct text column across differently structured source files. After merging, empty rows are removed and the data is shuffled. A balanced version is produced by downsampling the majority class, giving a final dataset of 21,402 emails with equal class representation.

4.3 Model Training and Explainability

The training pipeline splits the data 80/20 using stratified sampling (17,121 training, 4,281 test samples), fits a TF-IDF vectoriser on the training set only to prevent data leakage, and trains both classifiers. Models are saved using `joblib` and loaded on demand by other components. The Streamlit app uses `@st.cache_resource` to load the model only once per session.

The explainability module provides a single `explain_email()` function with three methods applied in priority order: SHAP first (using a cached `shap.LinearExplainer` built from a 200-email background sample), then LIME, then linear weight analysis as a fallback. All three methods return results in the same structured format, meaning the rest of the system works identically regardless of which explanation method was used.

4.4 Attacker Simulation Design

The attacker simulation tests model robustness by applying four text transformation rules to base phishing emails and checking whether the model still detects them.

Rule	Description
<code>urgency</code>	Adds time pressure language
<code>spoof_bank</code>	Spoofed bank security notice
<code>spoof_revenue</code>	Spoofed tax authority message
<code>fake_link</code>	Injects a malicious URL

Table 4.3: Attack rules used in the simulation.

Rules are applied individually and in pairs, giving 4 single-rule and 12 two-rule combinations per base email. The simulation runs across three thresholds (0.5, 0.6, 0.7). For any email that bypasses detection, a LIME explanation is automatically logged. Results feed into `analyse_simulation_results.py` and `visualise_results.py` to generate the summary tables and charts in Chapter 6.

4.5 MITRE ATT&CK Mapping

The mapping uses a scoring system where each of five techniques is associated with URL patterns (1 point each) and phishing keywords (2 points each). The email is assigned to the highest scoring technique, with T1566.001 as a fallback when nothing matches. The five techniques covered are T1566.001, T1566.002, T1598.001, T1598.002, and T1598.003. The mapping accuracy is formally evaluated in Chapter 6 against a manually labelled validation subset.

4.6 Reproducibility and CI

`src/reproduce_pipeline.py` runs all pipeline stages from a single command with a fixed random seed, saving a JSON metadata file per run. A GitHub Actions workflow (`.github/workflows/ci.yml`) automatically runs the pytest test suite on every push. Shared constants including `TFIDF_MAX_FEATURES=5000`, `DEFAULT_THRESHOLD=0.5`, and `RANDOM_SEED=42` are centralised in `src/utils.py`.

Chapter 5

Implementation

5.1 Overview

This chapter describes how the phishing detection system was built in Python across fifteen self contained modules. Each component was developed and tested independently before integration. Code listings are available in Appendix A and in the project repository.

5.2 Dataset Builder

The dataset builder (`src/build_dataset.py`) merges four CSV files from `data/raw` into a single structured dataset. The main implementation challenge I encountered was that each source uses different column names for the email content. Rather than hardcoding a fix per source, I used a `pick_text_column` function that checks against a list of common names and falls back to any string-typed column if none match. Labels are assigned (0 for legitimate, 1 for phishing), empty rows are dropped, and the combined data is shuffled. A balanced version is produced by downsampling the majority class. Both datasets are saved to `data/processed/`.

5.3 Baseline Classifier

The classifier (`src/mvp_baseline.py`) combines a TF-IDF vectoriser with Logistic Regression. The vectoriser is fitted only on the training data to prevent data leakage, configured with 5,000 features, lowercase normalisation, and English stopwords removal. Logistic Regression is trained with `class_weight="balanced"` to handle any remaining class imbalance. A Multinomial Naive Bayes model is trained alongside for comparison. Both models are saved using `joblib` for reuse across all other components. After training, SHAP global and local explanations are also computed as part of the training run, discussed further in Section 5.4.

5.4 Explainability Integration

Explainability was implemented in `src/xai_explainer.py` using three methods applied in priority order through a single `explain_email()` function.

SHAP is the first method, used when `use_shap=True`. A `shap.LinearExplainer` is built from a 200-email background sample and cached after the first call. It assigns

each word a contribution value based on how much it shifts the prediction away from the baseline, computed efficiently for linear models without full Shapley enumeration.

```
def _get_shap_linear_explainer():
    background_texts = text_series.head(200).tolist()
    X_background = vectorizer.transform(
        background_texts
    )
    _shap_linear_explainer = shap.LinearExplainer(
        clf, X_background
    )
    return _shap_linear_explainer
```

LIME is the second method, used by default when `use_lime=True`. It generates perturbations of the input email, fits a local interpretable model around the prediction, and returns the top contributing words with weights. LIME requires access to `predict_proba` rather than just `predict`, so the probability output for both classes is exposed.

Linear weight analysis is the fallback, computing importance directly by multiplying each word's TF-IDF value by its Logistic Regression coefficient. This runs in under 1 ms and requires no additional libraries. All three methods return results in the same structured format.

5.5 Rigorous Evaluation and Error Analysis

`src/evaluate_models_rigorously.py` runs repeated stratified cross-validation (5 folds, 3 repeats) for both models, producing mean scores with 95% confidence intervals. It also runs threshold sensitivity analysis across 0.10 to 0.95 and assesses calibration using Brier score and ECE.

`src/evaluate_mitre_mapping.py` formally evaluates the MITRE mapping against a manually labelled validation subset, computing primary-label accuracy, macro F1, exact match rate, and micro precision, recall, and F1.

`src/run_error_analysis.py` reconstructs the test split and generates seven output files covering all predictions, false positives, false negatives, the fifty most confidently wrong cases in each direction, and the most common vocabulary in each error type.

5.6 Attacker Simulation

The attacker simulation (`src/attacker_sim.py`) applies four text transformation rules to base phishing emails and tests whether the model still detects them. Rules are applied individually and in pairs, producing 4 single-rule and 12 two-rule combinations per email. The key design decision is the variant generation logic, which ensures rule order is preserved in chained attacks (rule A then B is a different variant to rule B then A):

```
def generate_attack_variants(base_text):
    variants = []
    for name, fn in ATTACK_RULES.items():
        variants.append({
            "attack_type": "single_rule",
```

```

        "rule_chain": name,
        "modified_text": fn(base_text),
    })
    for n1, f1 in ATTACK_RULES.items():
        for n2, f2 in ATTACK_RULES.items():
            if n1 == n2:
                continue
            variants.append({
                "attack_type": "multi_rule",
                "rule_chain": f"{n1}+{n2}",
                "modified_text": f2(f1(base_text)),
            })
    return variants

```

The simulation runs across three thresholds (0.5, 0.6, 0.7). For any email that bypasses detection, a LIME explanation is automatically logged, providing insight into what vocabulary allowed the attack to succeed.

5.7 MITRE ATT&CK Mapping

The mapping in `src/attacker_sim.py` uses a scoring system where URL pattern matches score 1 point and phishing keyword matches score 2 points per technique. The email is assigned to the highest scoring technique across five supported ones, with T1566.001 as a fallback when nothing matches. Pattern and keyword definitions are listed in Appendix A.

5.8 Streamlit Dashboard

The dashboard (`src/app.py`) accepts email text or uploaded `.eml` files and returns a prediction, confidence score, MITRE mapping, and XAI explanation. The sidebar provides model selection, a threshold slider (0.10–0.95), a feature count slider, and checkboxes for LIME and SHAP. SHAP is labelled as experimental in the interface. The `.eml` parsing in `src/email_ingestion.py` handles plain text, HTML-only emails (stripping scripts and styles), and multipart emails, reporting attachment filenames without reading their content. The model is loaded once on startup using `@st.cache_resource`.

5.9 Testing

The test suite in `tests/` covers four areas: `test_utils.py` tests `classify_email()` and shared helpers; `test_xai_explainer.py` tests all three explanation methods and fallback behaviour; `test_email_ingestion.py` tests `.eml` parsing including HTML and attachments; and `test_stage_scripts_helpers.py` tests helper functions from the evaluation and analysis scripts. A shared `conftest.py` provides pre-fitted fixtures and custom markers. A GitHub Actions workflow runs the test suite on every push.

```
python -m pytest tests/ -v
```

Chapter 6

Experimental Results and Evaluation

This chapter presents the evaluation of the phishing detection system, covering baseline model performance, cross-validation stability, threshold sensitivity, calibration, explainability outputs, attacker simulation results, and the formal MITRE mapping evaluation.

6.1 Baseline Model Performance

6.1.1 Classification Results

The Logistic Regression model achieved 93.76% accuracy on the held-out test set of 4,281 emails, with balanced precision and recall across both classes.

Class	Precision	Recall	F1-score	Support
Legitimate (0)	0.94	0.94	0.94	2141
Phishing (1)	0.94	0.94	0.94	2140
Accuracy			0.94	4281
Macro Avg	0.94	0.94	0.94	4281
Weighted Avg	0.94	0.94	0.94	4281

Table 6.1: Classification report for the Logistic Regression baseline model on the held-out test set.

I believe consistency across both classes is the most notable aspect of these results. Precision and recall are almost identical for legitimate and phishing emails, suggesting the model is not biased toward one class. This matters because both error types carry real consequences: missed phishing emails mean attacks get through, while false alarms disrupt legitimate traffic. The ROC AUC of 0.9808 confirms strong discriminatory power across all possible thresholds.

6.1.2 Confusion Matrix and Error Analysis

	Predicted Legitimate	Predicted Phishing
Actual Legitimate	2002	139
Actual Phishing	128	2012

Table 6.2: Confusion matrix for the baseline model.

The model produced 139 false positives and 128 false negatives from 4,281 test emails, an overall error rate of 6.24%. The false negative rate within the phishing class is 5.98% and the false positive rate within the legitimate class is 6.49%, both broadly equal and reflecting the balanced training data.

The error analysis module identified the most common vocabulary in each error type. Top false positive tokens include **account** (185), **bank** (156), **money** (146), and **email** (157) which are financial vocabulary the model associates with phishing that also appears in legitimate business communications. False negative emails are dominated by common function words with no distinctive phishing vocabulary, meaning the model had nothing strong to react to. The hardest false negative was classified as legitimate with 94.8% confidence despite containing a URL; its content reads as promotional spam rather than targeted phishing, suggesting a possible labelling issue in the original dataset.

6.1.3 ROC Curve Comparison

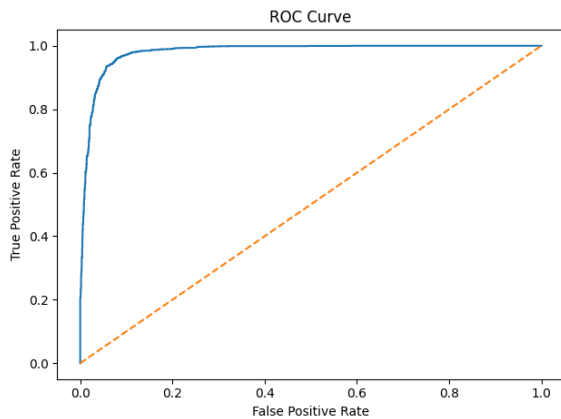


Figure 6.1: ROC curve: Logistic Regression (AUC = 0.9808).

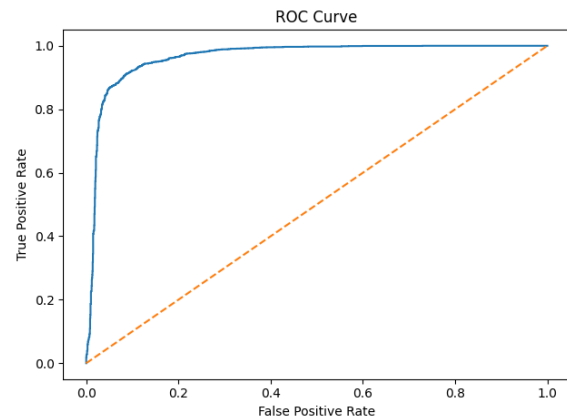


Figure 6.2: ROC curve: Naive Bayes (AUC = 0.9642).

The Logistic Regression curve rises almost vertically from the origin, achieving a very high true positive rate while keeping the false positive rate extremely low. The Naive Bayes curve shows a more gradual initial climb, requiring a higher false positive rate to reach the same detection level. This visually confirms that Logistic Regression is the stronger classifier and supports the model selection decision.

6.2 Cross-Validation and Rigorous Evaluation

6.2.1 Cross-Validation Stability

Repeated stratified cross-validation (5 folds, 3 repeats, 15 runs per model) confirmed that the single-split results are stable.

Metric	LR Mean	LR 95% CI	NB Mean	NB 95% CI
Accuracy	0.938	[0.933, 0.941]	0.909	[0.901, 0.917]
Precision	0.937	[0.934, 0.941]	0.905	[0.895, 0.916]
Recall	0.939	[0.931, 0.945]	0.914	[0.905, 0.923]
F1-Score	0.938	[0.933, 0.942]	0.909	[0.902, 0.917]
AUC	0.980	[0.978, 0.982]	0.963	[0.959, 0.967]

Table 6.3: Cross-validation summary (5 folds $\times$ 3 repeats = 15 runs).

The narrow confidence intervals confirm consistent performance across partitions. The LR AUC CI of [0.978, 0.982] shows very low variance. Logistic Regression outperforms Naive Bayes across all five metrics, confirming the model selection decision.

6.2.2 Threshold Sensitivity and Calibration

Threshold sensitivity was tested from 0.10 to 0.95. The optimal threshold by F1-score is **0.40** (F1=0.940), marginally better than the default 0.50 (F1=0.938). The model is slightly conservative — a small reduction in threshold catches more phishing with only a modest increase in false positives. The default of 0.50 was retained for the simulation to keep results interpretable across the three thresholds tested.

Threshold	Precision	Recall	F1	Accuracy
0.10	0.721	0.999	0.837	0.806
0.30	0.886	0.982	0.932	0.928
0.40	0.918	0.964	0.940	0.939
0.50	0.937	0.939	0.938	0.938
0.60	0.951	0.906	0.928	0.930
0.70	0.960	0.859	0.907	0.912
0.90	0.978	0.614	0.755	0.800

Table 6.4: Threshold sensitivity for Logistic Regression. Best F1 at threshold 0.40.

The Brier score of 0.052 and ECE of 0.056 confirm the model’s probability outputs are well calibrated. At the 0.9–1.0 confidence range, 97.8% of emails are genuinely phishing; at 0.0–0.1, only 0.18% are. The probability score can therefore be interpreted as an actual likelihood rather than just a ranking signal.

6.3 Explainability Results

6.3.1 LIME Explanations

LIME was applied to the first test email (the Reza B’Far email) to examine what the model reacted to. The top contributing words were 2002 (+0.0875), wrote (+0.0865), beberg (+0.0490), Sep (+0.0467), and Java (+0.0428). The word management received a negative weight (-0.0343), pushing toward legitimate.

None of these are genuine phishing indicators. The model was responding to dataset-specific patterns — the date, reply chain vocabulary, and a person’s name — rather than phishing signals. This illustrates why explainability tools are useful for catching cases where the model makes decisions for the wrong reasons.

6.3.2 SHAP Global Feature Importance

```

SHAP EXPLANATIONS (Logistic Regression)
=====

Building SHAP explainer (using a small background sample)...

Computing SHAP global feature importance...
Using a sample of 2000 emails for global SHAP.

Top 20 global features by SHAP (sample-based):
  enron: 0.199871
  ect: 0.139632
  2002: 0.137035
  linux: 0.124613
  account: 0.114097
  wrote: 0.090029
  http: 0.089064
  click: 0.075744
  listinfo: 0.072991
  20: 0.065763
  fork: 0.064679
  bank: 0.063821
  mailman: 0.062954
  org: 0.058171
  vince: 0.054474
  money: 0.052556
  www: 0.050181
  lists: 0.049422
  list: 0.049399
  spamassassin: 0.048848

Computing SHAP local explanation for test index 0...
True label for this email: phishing

Top 10 local SHAP features for this email:
  memory: 0.611170
  beberg: 0.600919
  wrote: 0.334555
  2002: 0.306237
  java: 0.282853

```

Figure 6.3: Top 20 global SHAP feature importance scores computed across 2000 training emails.

SHAP computed mean absolute feature contributions across 2000 training emails. The highest scoring features include `enron` (0.200), `ect` (0.140), `2002` (0.137), `linux` (0.125), `vince` (0.054), and `spamassassin` (0.049) — all dataset-specific terms from the Enron corpus or SpamAssassin collection rather than genuine phishing vocabulary. Legitimate phishing signals do appear: `account` (0.114), `http` (0.089), `click` (0.076), and `bank` (0.064) are all present. However, the prominence of dataset artefacts confirms that the model has learned to distinguish between two specific email corpora as much as it has

learned to detect phishing [3]. The local SHAP explanation for the same email showed **memory** (0.611) and **beberg** (0.601) as top contributors, consistent with the LIME output and giving confidence that both explanations are stable.

6.3.3 Streamlit Dashboard

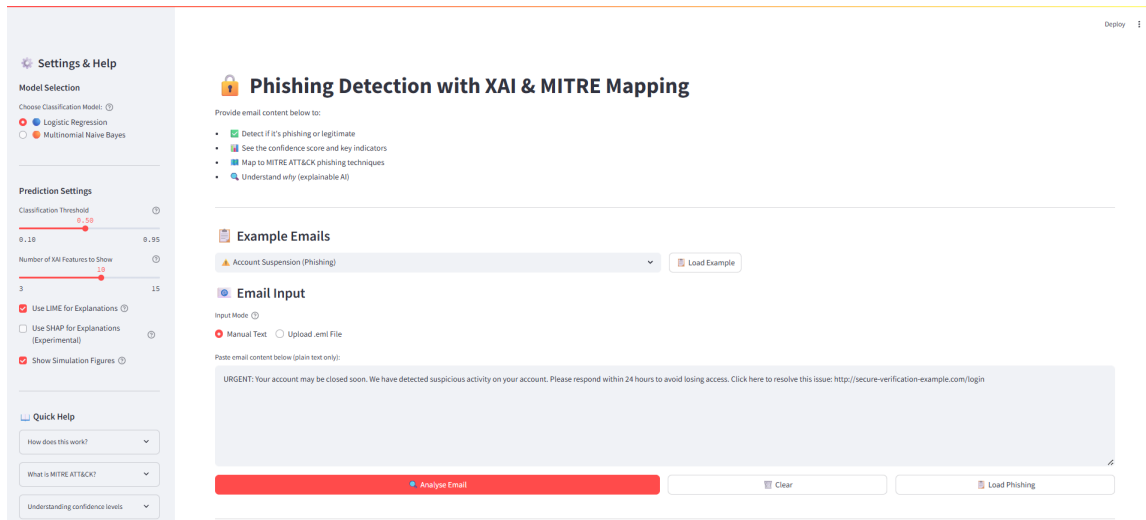


Figure 6.4: The Streamlit dashboard showing the input screen with an example phishing email loaded.

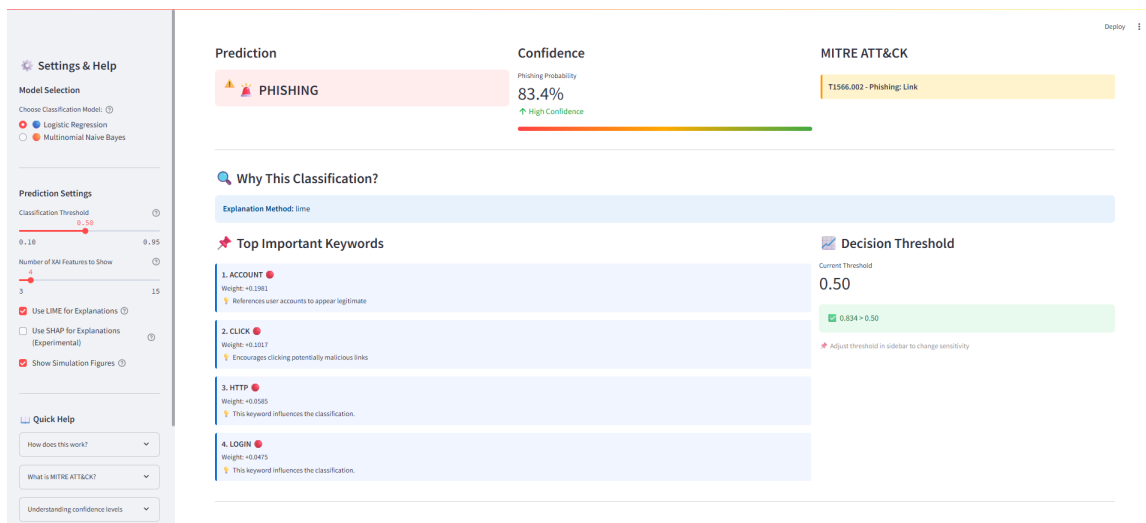


Figure 6.5: Analysis results showing 83.4% phishing confidence, T1566.002 MITRE mapping, and LIME explanation for the account suspension email.

The account suspension email was correctly classified as phishing with 83.4% confidence and mapped to T1566.002 - **Phishing: Link**. The LIME explanation highlighted ACCOUNT (+0.1981), CLICK (+0.1017), HTTP (+0.0585), and LOGIN (+0.0475) — all genuinely relevant phishing signals rather than dataset artefacts, giving more confidence in this prediction than in cases where Enron-specific terms dominate.

6.4 Attacker Simulation Results

An attacker–defender simulation tested the model against modified phishing emails across three thresholds (0.5, 0.6, 0.7), four single-rule attacks, and twelve two-rule combinations.

6.4.1 Effect of Classification Threshold

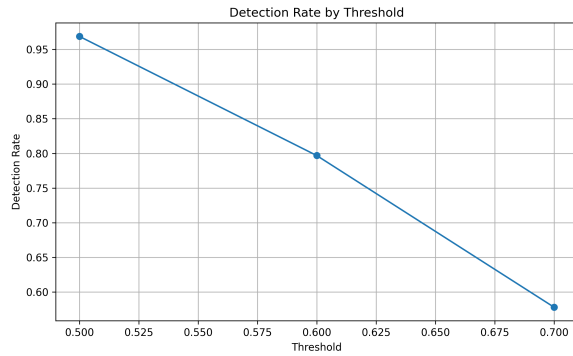


Figure 6.6: Detection rate by threshold.

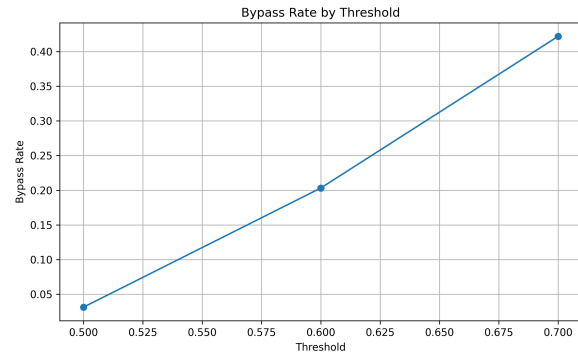


Figure 6.7: Bypass rate by threshold.

At threshold 0.5, the model achieves a detection rate of 0.969, accuracy of 0.971, F1 of 0.984, and bypass rate of 0.031. Moving to 0.6, detection drops to 0.844 and bypass rises to 0.156. At 0.7, detection falls to 0.578 and bypass reaches 0.422 — over four in ten attacks get through. A key finding from the cross-tabulation is that at threshold 0.5, every multi-rule attack is detected (detection rate 1.0), and degradation only appears at higher thresholds.

6.4.2 Detection Rate by Attack Type and Rule Chain

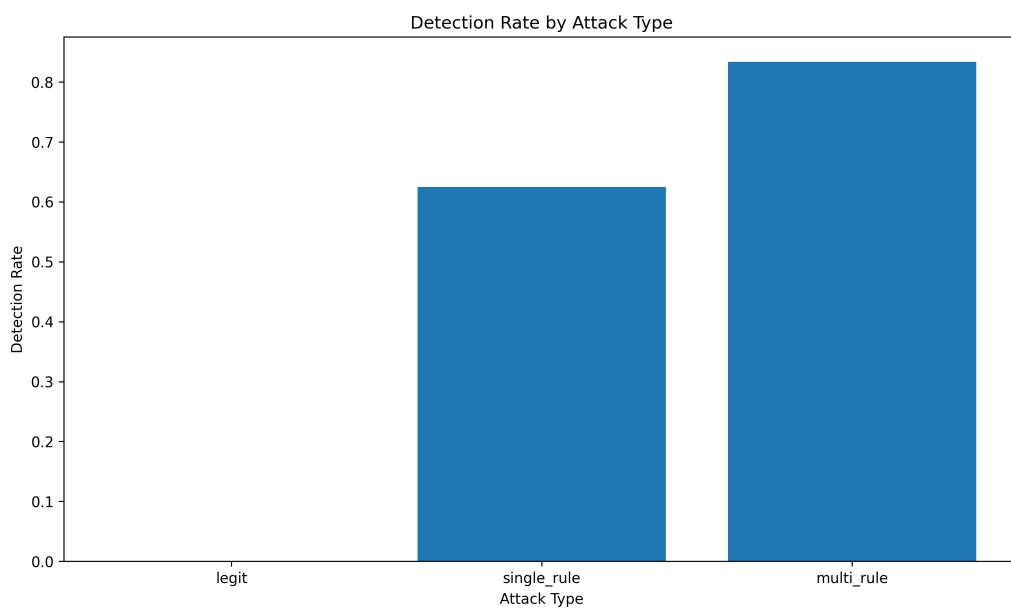


Figure 6.8: Detection rate by attack type.

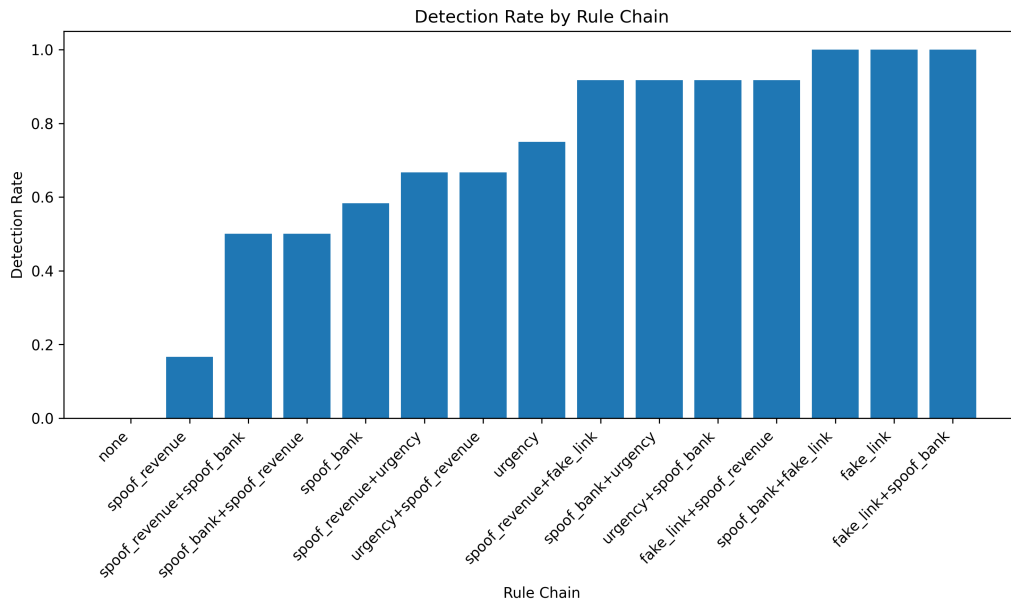


Figure 6.9: Detection rate by rule chain.

Single-rule attacks achieved a detection rate of 0.646 while multi-rule attacks reached 0.847. The gap is counterintuitive: a more aggressive attacker combining two techniques introduces more suspicious vocabulary and is therefore easier to detect.

The rule chain breakdown reveals the extremes. **fake_link** alone achieves 100% detection and any chain including it achieves at least 0.917. **spoof_revenue** alone achieves just 0.167, producing a phishing probability of only 0.447 — below the 0.5 threshold. LIME explained why: **eligible** (-0.084) and **Revenue** (-0.034) were being treated as legitimate indicators. Any chain combining **spoof_revenue** and **spoof_bank** without a URL reaches only 0.500, since neither rule introduces URL-based vocabulary.

6.4.3 Detection Rate by MITRE Technique

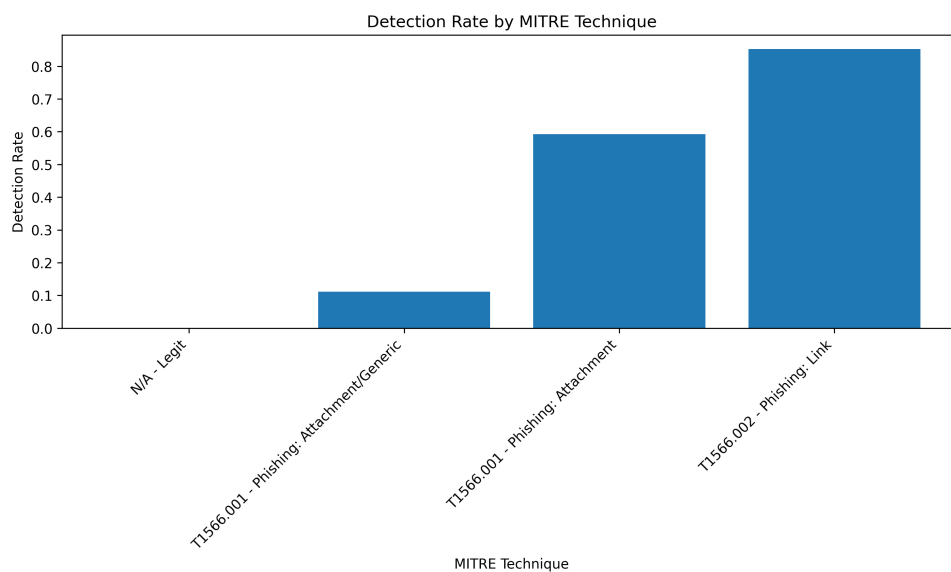


Figure 6.10: Detection rate by MITRE ATT&CK technique.

T1566.002 - Phishing: Link achieves the highest detection rate at 0.853. T1566.001 - Phishing: Attachment reaches 0.704. The generic fallback category T1566.001 - Attachment/Generic, used when no patterns match, has a detection rate of just 0.111 and a bypass rate of 0.889. Emails that trigger no MITRE pattern matches are almost completely invisible to the model.

6.5 MITRE ATT&CK Mapping Evaluation

The mapping was formally evaluated against a manually labelled validation subset.

Metric	Score
Primary-label Accuracy	0.933
Macro Precision	0.950
Macro Recall	0.933
Macro F1	0.931
Multi-label Exact Match Rate	0.267
Multi-label Micro Precision	0.593
Multi-label Micro Recall	0.941
Multi-label Micro F1	0.727

Table 6.5: MITRE mapping evaluation against a manually labelled validation subset (15 samples).

Primary-label accuracy of 93.3% shows the mapping correctly identifies the dominant technique in most cases. T1566.002, T1598.002, and T1598.003 achieved perfect F1. T1566.001 achieved F1=0.857 with precision of 0.75, occasionally over assigning. T1598.001 was hardest at F1=0.800 with recall of 0.667. The multi-label exact match rate of 26.7% is low, but the micro recall of 0.941 is the more important figure, the system rarely misses a technique that is present. The validation subset of only 15 samples limits confidence in these figures; they should be treated as indicative rather than definitive.

Chapter 7

Discussion

7.1 Trust and Transparency in AI-Based Phishing Detection

One of the central goals of my project was to make the model’s decisions understandable rather than just accurate. The LIME explanation for the account suspension email highlighted ACCOUNT, CLICK, HTTP, and LOGIN as the top contributing words, which are all genuinely relevant phishing signals. The agreement between SHAP, LIME, and linear weight analysis on the same email gives confidence that the explanations are consistent rather than arbitrary.

That said, explainability does not automatically mean the model is right. The baseline evaluation surfaced a case where the model flagged an email with 90.7% phishing confidence despite the content reading like a technical software discussion. The LIME output showed 2002, wrote, beberg, and Sep as the top contributors, none of which are genuine phishing indicators. The model was responding to Enron-specific patterns rather than anything resembling a phishing attempt. This is exactly the kind of case a pure accuracy metric would hide entirely.

The SHAP global feature importance analysis made this problem visible at scale. Several of the top 20 features such as `enron` (0.200), `ect` (0.140), `2002` (0.137), `linux` (0.125), `vince` (0.054), and `spamassassin` (0.049) are dataset-specific terms rather than genuine phishing signals. The model has learned to distinguish the Enron corpus from the phishing datasets as much as it has learned to detect actual phishing language, a direct consequence of using legacy datasets [3]. Without SHAP this would not have been visible from the classification report alone, which is perhaps the strongest argument for including explainability in this kind of project.

I also believe LIME’s instability is worth acknowledging. Running it twice on the same email can produce different results due to random perturbations. Explanations should be treated as a guide rather than a definitive account of the model’s reasoning [13]. SHAP does not have this problem given the cached background sample approach used here, which is one reason to prefer it when consistency matters.

7.2 Strengths of the Proposed System

The strongest result is the attacker simulation rather than the baseline accuracy. At threshold 0.5, the model achieved a detection rate of 0.969 and a bypass rate of 0.031. The

cross-tabulation revealed that at this threshold every single multi-rule attack was detected, a detection rate of 1.0 for that category. Any chain including `fake_link` achieved 100% detection, confirming that URL-based vocabulary is a strong and reliable phishing signal in this model.

The cross-validation AUC confidence interval of [0.978, 0.982] across 15 evaluation runs confirms the model's performance is stable rather than a product of a favourable split. The MITRE mapping evaluation gave a primary-label accuracy of 93.3% and multi-label micro recall of 94.1%, and the mapping rarely misses a technique that is present. In a security context where missing a technique is more costly than over flagging, this is the better side to be on. The Brier score of 0.052 and ECE of 0.056 confirm probability outputs are well calibrated, which matters because the threshold slider and simulation both rely on scores being interpretable.

7.3 Limitations of the Current Approach

In my opinion the clearest limitation is the model's vulnerability to phishing emails that avoid obvious indicators. The `spoof_revenue` single-rule attack achieved a bypass rate of 83.3% and the generic MITRE fallback had a detection rate of just 0.111. These both point to the same problem: TF-IDF has no understanding of intent, only vocabulary. The LIME explanation for a `spoof_revenue` bypass made this concrete — `eligible` (-0.084) and `Revenue` (-0.034) were treated as legitimate indicators, dropping the phishing probability to 0.447. The model was not making a random error; it was responding to patterns learned from Enron business emails where tax related vocabulary is associated with legitimate communication.

This compounds the dataset artefact problem revealed by SHAP. A model where `enron` is the most important global feature has learned to distinguish two specific email corpora from the early 2000s rather than to detect phishing in any general sense. The cross-validation confirms the performance is real and stable, but it should not be expected to transfer to a different organisation's email traffic without retraining. This is not a hypothetical concern: LLM-generated phishing already produces fluent, context-aware content that word-frequency approaches are not built to handle [2, 1].

The attacker simulation is also constrained by its rule-based design. A real attacker can adapt in ways that four fixed rules cannot replicate. The bypass rates observed here should be considered a floor rather than a ceiling for what a motivated attacker could achieve.

7.4 Implications for Real-World Deployment

The model works well as a first pass filter. A detection rate of 0.969 at threshold 0.5 and 93.76% baseline accuracy suggest it would catch a meaningful proportion of phishing attempts. The LIME and SHAP explanations give analysts something concrete to review beyond a probability score.

The threshold finding is relevant for deployment. Threshold 0.40 gives marginally better F1, but the right choice depends on organisational priorities. Moving from 0.5 to 0.7 in the simulation nearly halved the detection rate, while moving to 0.4 catches more phishing at the cost of slightly more false positives. Well calibrated probability outputs mean this decision is at least meaningful rather than arbitrary.

The bypass rate against subtle social engineering makes clear this system should not be the only line of defence. An attacker who avoids URL-based vocabulary has a bypass rate of over 83%. Real deployment would need to combine this model with URL analysis, sender reputation checks, or a more context-aware classifier.

The dataset age is the most important practical concern. As LLM-generated phishing becomes more common [1], patterns from 2002 will become less useful. The SHAP analysis provided direct evidence: when `enron` is the most important global feature, the model is working from signals that no longer exist in modern email traffic.

Overall the system demonstrates that combining a simple, interpretable model with multiple explainability tools and adversarial testing produces something more useful and more honest than a classification report alone would suggest. The SHAP dataset artefact finding in particular is a genuine contribution beyond what the accuracy figures alone would show.

Chapter 8

Conclusion and Future Work

8.1 Summary of the Project

This project built a phishing email detection system that explains its decisions rather than just producing a classification. The motivation came from a gap in the literature: only 39% of phishing detection studies implement any explainability [3], limiting how useful these systems are in practice.

The system uses Logistic Regression with TF-IDF on a balanced dataset of 21,402 emails, achieving 93.76% accuracy confirmed stable by repeated stratified cross-validation (AUC CI [0.978, 0.982]). Three explanation methods were integrated: SHAP for global feature importance, LIME for per-email explanations, and linear weights as a fallback. An attacker simulation tested the model across four attack rules and twelve two-rule combinations, with results mapped to five MITRE ATT&CK techniques and the mapping evaluated. A dedicated error analysis module examined false positives and false negatives, and a Streamlit dashboard made the system interactive.

8.2 Achievement of Objectives

All objectives were met. The model was evaluated not just on a train/test split but through repeated cross-validation and an attacker simulation. At threshold 0.5, the simulation achieved a detection rate of 0.969 with every multi-rule attack detected, a result invisible to the classification report alone.

The SHAP global analysis was the most important finding beyond the headline accuracy figures. Top features included `enron` (0.200), `ect` (0.140), and `2002` (0.137) — dataset-specific terms rather than genuine phishing vocabulary. This provides direct evidence of the legacy dataset problem identified in the technology review, and is a contribution that accuracy figures alone could not have revealed.

The rule chain results showed `fake_link` achieving 100% detection while `spoof_revenue` reached only 16.7%. The LIME explanation for a `spoof_revenue` bypass confirmed why: `eligible` (-0.084) and `Revenue` (-0.034) were treated as legitimate indicators, dropping the probability to 0.447. The MITRE mapping evaluation produced 93.3% primary-label accuracy and 94.1% multi-label micro recall. Threshold sensitivity identified 0.40 as the optimal F1 point, and calibration scores (Brier 0.052, ECE 0.056) confirmed probability outputs are meaningful.

8.3 Limitations

The core limitation is TF-IDF’s inability to detect phishing intent when suspicious vocabulary is absent. The `spoof_revenue` bypass rate of 83.3% and the 11.1% detection rate for the generic MITRE fallback both reflect this. The SHAP finding that `enron` is the top global feature confirms the model is partly distinguishing between two dated corpora rather than detecting phishing in a general sense. The attacker simulation’s fixed rules mean bypass rates should be considered a floor rather than a ceiling. LIME’s instability means per-email explanations should be treated as guidance rather than ground truth [13].

8.4 Recommendations for Future Work

8.4.1 Comparison with Deep Learning Models

Testing BERT or DistilBERT on the same evaluation pipeline would answer whether transformer accuracy improvements hold up in the attacker simulation, and what SHAP explanations look like for a transformer versus Logistic Regression. The literature shows BERT dropping from 96.7% to 89.3% on LLM-generated phishing [2], which raises genuine questions about whether the benchmark performance gap over simpler models is as large as it appears.

8.4.2 Adversarial Attack Testing

Future work could implement gradient-based adversarial attacks, where the attacker has knowledge of the model and crafts inputs to push predictions toward legitimate. This would give a harsher and more realistic test of robustness than predefined rules.

8.4.3 URL and Metadata Features

Adding URL analysis such as domain age, redirect chains, and phishing blacklists, would directly address the `spoof_revenue` weakness. Sender metadata such as SPF and DKIM validation are signals TF-IDF cannot capture from the email body alone.

8.4.4 REST API Deployment

Wrapping the model in a REST API would allow testing against real incoming email traffic, surfacing edge cases and failure modes that static dataset evaluation cannot anticipate.

8.4.5 Dataset Modernisation

Sourcing a more recent phishing dataset — ideally including LLM-generated examples — and rerunning the same SHAP analysis would allow a direct comparison of whether the dataset artefact problem persists or diminishes, which would be a useful finding in its own right [3].

Appendix A

Code Excerpts and Repository

The project codebase is publicly available at:

`https://github.com/NathanCarr10/fyp-phishing-detection-mitre-xai`

A screencast demonstration of the system is available at:

`https://youtube.com/watch?v=CVL3C\_xHcic`

Installation and Running

Requirements: Python 3.8 or higher.

```
pip install -r requirements.txt
python src/reproduce_pipeline.py --seed 42
.\run_reproducible_pipeline.cmd          # Windows
python -m streamlit run src/app.py
python -m pytest tests/ -v
```

To skip dataset building if processed files exist:

```
python src/reproduce_pipeline.py --seed 42
--skip-build-dataset
```

Source Files

File	Description
<code>build_dataset.py</code>	Loads raw CSVs, detects text columns, assigns labels, merges, shuffles, and produces balanced output
<code>mvp_baseline.py</code>	Trains both models, evaluates on test set, generates ROC curves, runs LIME and SHAP, saves models
<code>models.py</code>	Reusable model factory and evaluation helpers
<code>xai_explainer.py</code>	SHAP (priority), LIME (default), linear weights (fallback)
<code>attacker_sim.py</code>	Attacker simulation across four rules, twelve combinations, and three thresholds; implements MITRE mapping
<code>analyse_simulation_results.py</code>	Processes simulation log into seven summary CSVs
<code>visualise_results.py</code>	Generates seven PNG charts from analysis CSVs
<code>compare_models.py</code>	Side-by-side model comparison
<code>evaluate_models_rigorously.py</code>	Cross-validation, threshold sensitivity, Brier score, ECE
<code>evaluate_mitre_mapping.py</code>	MITRE mapping evaluation
<code>run_error_analysis.py</code>	False positive and negative analysis, top error tokens
<code>reproduce_pipeline.py</code>	Pipeline runner with seed control
<code>email_ingestion.py</code>	.eml parsing with HTML-to-text conversion
<code>utils.py</code>	Shared constants and helpers
<code>app.py</code>	Streamlit dashboard

Table A.1: Source files in the `src/` directory.

Test Suite

File	Description
<code>conftest.py</code>	Shared fixtures and custom pytest markers
<code>test_utils.py</code>	Tests for <code>classify_email()</code> , label mapping, constants, and directory utilities
<code>test_xai_explainer.py</code>	Tests for all three explanation methods and fallback behaviour
<code>test_email_ingestion.py</code>	Tests for .eml parsing: plain text, HTML, attachments, empty input
<code>test_stage_scripts_helpers.py</code>	Tests for helpers from evaluation and analysis scripts

Table A.2: Test files in the `tests/` directory.

Dataset Builder — Column Detection

```
TEXT_COLUMN_CANDIDATES = [
    "text", "email", "body", "content",
    "message", "email_text", "mail", "data"
]

SOURCES = [
    ("Enron.csv", 0), # legitimate
    ("SpamAssasin.csv", 1), # phishing
    ("Nazario.csv", 1), # phishing
    ("Nigerian_Fraud.csv", 1), # phishing
]

def pick_text_column(df):
    for c in TEXT_COLUMN_CANDIDATES:
        if c in df.columns:
            return c
    for c in df.columns:
        if df[c].dtype == "object":
            return c
    raise ValueError(
        f"No text column found: {df.columns}"
    )
```

Model Training and Saving

```
def vectorize_text(X_train, X_test):
    vectorizer = TfidfVectorizer(
        lowercase=True,
        stop_words="english",
        max_features=5000,
    )
    X_train_tfidf = vectorizer.fit_transform(X_train)
    X_test_tfidf = vectorizer.transform(X_test)
    return vectorizer, X_train_tfidf, X_test_tfidf

clf = LogisticRegression(
    max_iter=1000,
    class_weight="balanced"
)
clf.fit(X_train_tfidf, y_train)

joblib.dump(vectorizer, "models/tfidf_vectorizer.joblib")
joblib.dump(clf, "models/logreg_model.joblib")
```

SHAP Explainer

```
def _get_shap_linear_explainer():
    global _shap_linear_explainer
    if _shap_linear_explainer is not None:
        return _shap_linear_explainer
    vectorizer, clf = _get_model()
    background_texts = (
        dataset[TEXT_COLUMN].dropna()
        .astype(str).head(200).tolist()
    )
    X_background = vectorizer.transform(
        background_texts
    )
    _shap_linear_explainer = shap.LinearExplainer(
        clf, X_background
    )
    return _shap_linear_explainer
```

explain_email() Public API

```
def explain_email(
    text: str,
    num_features: int = 10,
    threshold: float = 0.5,
    use_lime: bool = True,
    use_shap: bool = False,
) -> Dict[str, Any]:

    X = vectorizer.transform([text])
    proba = clf.predict_proba(X)[0]
    phishing_prob = float(proba[phishing_index])
    pred_label = 1 if phishing_prob >= threshold \
        else 0

    if use_shap and _SHAP_AVAILABLE:
        features = _explain_with_shap(text)
        method = "shap"
    elif use_lime and _LIME_AVAILABLE:
        features = _explain_with_lime(text)
        method = "lime"
    else:
        features = _explain_with_linear_weights(text)
        method = "linear"

    return {
        "pred_label": int(pred_label),
        "phishing_probability": phishing_prob,
```

```

        "is_phishing":          bool(pred_label == 1),
        "top_features":        top_features_struct,
        "method":              method,
    }

```

Attack Rules and Variant Generation

```

ATTACK_RULES = {
    "urgency":          add_urgency,
    "spoof_bank":       spoof_bank,
    "spoof_revenue":    spoof_revenue,
    "fake_link":        add_fake_link,
}

def generate_attack_variants(base_text):
    variants = []
    for name, fn in ATTACK_RULES.items():
        variants.append({
            "attack_type":    "single_rule",
            "rule_chain":     name,
            "modified_text":  fn(base_text),
        })
    for n1, f1 in ATTACK_RULES.items():
        for n2, f2 in ATTACK_RULES.items():
            if n1 == n2:
                continue
            variants.append({
                "attack_type":    "multi_rule",
                "rule_chain":     f"{n1}+{n2}",
                "modified_text":  f2(f1(base_text)),
            })
    return variants

```

MITRE ATT&CK Mapping

```

def mitre_mapping(email_text, return_all=False):
    text_lower = email_text.lower()
    scores = {}
    for tech_id, info in MITRE_PATTERNS.items():
        score = sum(1 for p in info["patterns"]
                    if p in text_lower)
        score += sum(2 for k in info["keywords"]
                    if k in text_lower)
        if score > 0:
            scores[tech_id] = score
    if not scores:
        return "T1566.001 - Phishing: Attachment/Generic"

```



```
best = max(scores, key=scores.get)
return f"{best} - {MITRE_PATTERNS[best]['name']}"
```

Error Analysis Helpers

```
def tokenize(text: str) -> list[str]:
    return re.findall(r"[a-z]{3,}", text.lower())

def top_tokens(texts, top_n):
    counter = Counter()
    for text in texts:
        counter.update(tokenize(text))
    return counter.most_common(top_n)

def patch_legacy_logreg(clf) -> None:
    if not hasattr(clf, "multi_class"):
        clf.multi_class = "auto"
```

Appendix B

Additional Evaluation Results

Cross-Validation Summary

Table B.1 shows the full cross-validation results for both models across all five metrics, including standard deviations and 95% confidence intervals computed across 15 evaluation runs.

Model	Metric	Mean	Std	CI Low	CI High
Logistic Regression	Accuracy	0.9382	0.0023	0.9332	0.9414
Logistic Regression	Precision	0.9374	0.0024	0.9339	0.9414
Logistic Regression	Recall	0.9391	0.0044	0.9309	0.9455
Logistic Regression	F1-Score	0.9383	0.0024	0.9330	0.9416
Logistic Regression	AUC	0.9805	0.0014	0.9777	0.9819
Naive Bayes	Accuracy	0.9090	0.0048	0.9010	0.9167
Naive Bayes	Precision	0.9052	0.0061	0.8951	0.9160
Naive Bayes	Recall	0.9139	0.0060	0.9050	0.9232
Naive Bayes	F1-Score	0.9095	0.0047	0.9016	0.9169
Naive Bayes	AUC	0.9634	0.0026	0.9588	0.9669

Table B.1: Full cross-validation results (5 folds $\times$ 3 repeats = 15 runs). CI bounds are 2.5th and 97.5th percentiles across fold scores.

Model Comparison: Single-Split Results

Table B.2 shows the single-split test set results for both models, as reported by the training script.

Metric	Logistic Regression	Naive Bayes
Accuracy	0.9376	0.9089
Precision	0.9374	0.9052
Recall	0.9391	0.9139
F1-Score	0.9383	0.9095
AUC-ROC	0.9808	0.9642

Table B.2: Single-split test set comparison. Logistic Regression outperforms Naive Bayes across all five metrics.

Full Threshold Sensitivity

Table B.3 shows the complete threshold sensitivity results across all 18 thresholds tested from 0.10 to 0.95.

Threshold	Precision	Recall	F1	Accuracy
0.10	0.7209	0.9989	0.8374	0.8061
0.15	0.7800	0.9972	0.8753	0.8580
0.20	0.8295	0.9922	0.9036	0.8942
0.25	0.8628	0.9886	0.9214	0.9157
0.30	0.8865	0.9819	0.9317	0.9281
0.35	0.9046	0.9737	0.9379	0.9355
0.40	0.9184	0.9635	0.9404	0.9389
0.45	0.9284	0.9524	0.9403	0.9395
0.50	0.9374	0.9391	0.9383	0.9382
0.55	0.9447	0.9241	0.9343	0.9350
0.60	0.9510	0.9064	0.9282	0.9299
0.65	0.9564	0.8845	0.9191	0.9221
0.70	0.9604	0.8588	0.9068	0.9117
0.75	0.9642	0.8275	0.8906	0.8984
0.80	0.9687	0.7810	0.8648	0.8779
0.85	0.9726	0.7140	0.8235	0.8469
0.90	0.9784	0.6140	0.7545	0.8002
0.95	0.9845	0.3999	0.5688	0.6968

Table B.3: Full threshold sensitivity for Logistic Regression across all 18 thresholds. Optimal F1 is at threshold 0.40, highlighted in bold.

Attacker Simulation: Full Threshold Results

Table B.4 shows the complete simulation results at each of the three thresholds tested, across all 340 emails (320 phishing, 20 legitimate).

Threshold	TP	FP	TN	FN	Det.	Bypass	F1	Acc.
0.5	310	0	20	10	0.969	0.031	0.984	0.971
0.6	270	0	20	50	0.844	0.156	0.915	0.853
0.7	185	0	20	135	0.578	0.422	0.733	0.603

Table B.4: Simulation results by threshold. Precision is 1.000 at all thresholds — no legitimate emails were ever misclassified as phishing.

Attacker Simulation: Full Rule Chain Results

Table B.5 shows detection and bypass rates for all 16 rule chains tested, sorted by detection rate.

Rule Chain	Type	Det. Rate	Bypass Rate	F1
fake_link	single	1.000	0.000	1.000
fake_link+spoof_bank	multi	1.000	0.000	1.000
fake_link+urgency	multi	1.000	0.000	1.000
spoof_bank+fake_link	multi	1.000	0.000	1.000
urgency+fake_link	multi	1.000	0.000	1.000
fake_link+spoof_revenue	multi	0.917	0.083	0.957
spoof_bank+urgency	multi	0.917	0.083	0.957
spoof_revenue+fake_link	multi	0.917	0.083	0.957
urgency+spoof_bank	multi	0.917	0.083	0.957
urgency	single	0.833	0.167	0.909
spoof_revenue+urgency	multi	0.750	0.250	0.857
urgency+spoof_revenue	multi	0.750	0.250	0.857
spoof_bank	single	0.583	0.417	0.737
spoof_bank+spoof_revenue	multi	0.500	0.500	0.667
spoof_revenue+spoof_bank	multi	0.500	0.500	0.667
spoof_revenue	single	0.167	0.833	0.286

Table B.5: Full simulation results by rule chain, sorted by detection rate. Results are aggregated across all thresholds and simulation rounds.

Error Analysis Summary

Table B.6 shows the complete error analysis figures from the test set.

Metric	Value
Total test samples	4,281
Total errors	267
Error rate	6.24%
False positives	139
False negatives	128
True positives	2,012
True negatives	2,002
FP rate within legitimate class	6.49%
FN rate within phishing class	5.98%

Table B.6: Error analysis summary from the test set.

Table B.7 shows the top ten most frequent tokens appearing in false positive and false negative emails, excluding common function words.

False Positives		False Negatives	
Token	Count	Token	Count
account	185	com	95
email	157	one	92
bank	156	please	89
money	146	time	79
please	149	more	76
com	147	get	74
mail	137	they	74
any	141	here	74
was	124	out	70
all	174	from	131

Table B.7: Top distinctive tokens in false positive and false negative emails. FP tokens include financial vocabulary the model associates with phishing. FN tokens are common function words with no distinctive phishing signal.

Appendix C

Simulation Logs

The full attacker simulation log is available in the project repository at:

`simulation_results/attacker_simulation_log.csv`

Log File Columns

Column	Description
<code>timestamp</code>	UTC time of simulation run
<code>round</code>	Simulation round number
<code>threshold</code>	Classification threshold used
<code>base_index</code>	Index of base email used
<code>attack_type</code>	<code>single_rule</code> or <code>multi_rule</code>
<code>rule_chain</code>	Rule(s) applied
<code>modified_text</code>	Email text after transformation
<code>predicted_label</code>	0 = legitimate, 1 = phishing
<code>phishing_probability</code>	Raw model probability score
<code>attacker_success</code>	1 if email bypassed detection
<code>true_label</code>	Ground truth label
<code>mitre_technique</code>	Mapped ATT&CK technique
<code>xai_method</code>	Explanation method used (LIME for bypasses)
<code>xai_top_features</code>	Top LIME features logged for bypass cases

Table C.1: Column descriptions for the attacker simulation log CSV.

Representative Log Excerpt

Table C.2 shows a representative sample of simulation entries at threshold 0.5, illustrating both successful detections and attacker bypasses. The `spoof_revenue` bypass is included with its LIME explanation, showing how the model interpreted tax-related vocabulary as a legitimate indicator.

Rule Chain	Type	Prob.	Predicted	Bypass
urgency	single	0.634	Phishing	0
spoof_bank	single	0.512	Phishing	0
spoof_revenue	single	0.447	Legit	1
fake_link	single	0.743	Phishing	0
urgency+fake_link	multi	0.999	Phishing	0
spoof_revenue+spoof_bank	multi	0.487	Legit	1
fake_link+spoof_revenue	multi	0.917	Phishing	0

Table C.2: Representative simulation log excerpt at threshold 0.5. The `spoof_revenue` probability of 0.447 falls below the detection threshold. Any chain including `fake_link` produces probabilities at or near 1.0.

LIME Bypass Explanation

For the `spoof_revenue` bypass case, LIME logged the following top features:

Feature	Weight
account	+0.129
Irish	+0.070
tax	+0.066
refund	+0.039
follow	+0.018
eligible	-0.084
Revenue	-0.034
detected	-0.024
activity	-0.011
suspicious	+0.011

Table C.3: LIME feature weights for the `spoof_revenue` bypass email. Negative weights indicate words pushing the prediction toward legitimate. `eligible` and `Revenue` both reduce the phishing probability, explaining why the model missed this attack.

Cross-Tabulation: Threshold by Attack Type

Table C.4 shows detection rates broken down by both threshold and attack type, revealing that at threshold 0.5 all multi-rule attacks are detected.

Threshold	Attack Type	TP	FN	Det. Rate	Bypass
0.5	multi_rule	240	0	1.000	0.000
0.5	single_rule	70	10	0.875	0.125
0.6	multi_rule	220	20	0.917	0.083
0.6	single_rule	50	30	0.625	0.375
0.7	multi_rule	150	90	0.625	0.375
0.7	single_rule	35	45	0.438	0.563

Table C.4: Cross-tabulation of threshold and attack type. At threshold 0.5, every multi-rule attack is detected. Single-rule attacks are harder at every threshold.

Full Results by MITRE Technique

MITRE Technique	Total	TP	FN	Det. Rate
T1566.002 — Phishing: Link	780	665	115	0.853
T1566.001 — Phishing: Attachment	135	95	40	0.704
T1566.001 — Attachment/Generic	45	5	40	0.111
N/A — Legitimate	60	0	0	0.000

Table C.5: Simulation results by MITRE technique. The generic fallback category has a bypass rate of 88.9%, confirming that emails without detectable patterns are almost invisible to the model.

Appendix D

MITRE ATT&CK Mapping Tables

Techniques Implemented

Table D.1 shows the five MITRE ATT&CK techniques implemented in the mapping system, along with the URL patterns and phishing keywords used to score each one. Pattern matches score 1 point each and keyword matches score 2 points each. The email is assigned to the highest-scoring technique, with T1566.001 used as a fallback when nothing matches.

Technique	Patterns (1 pt each)	Keywords (2 pts each)
T1566.002 Phishing: Link	http://, https://, www., click here, .com/	verify, confirm, validate, urgent, immediate
T1566.001 Phishing: At- tachment	attached, document, in- voice, pdf, xlsx, docx	download, open, review, urgent
T1598.003 Spearphishing: Social Platform	linkedin, facebook, google, microsoft, apple	social, profile, connect, in- vite, update
T1598.001 Spearphishing: Attachment	invoice, receipt, contract, spreadsheet	sign, approve, review, ur- gent
T1598.002 Spearphishing: Shortened URL	bit.ly, tinyurl, goo.gl, short	shorten, track, click

Table D.1: MITRE ATT&CK techniques with associated patterns and keywords.

Formal Mapping Evaluation Results

Table D.2 shows the complete results from the formal evaluation of the MITRE mapping against the manually labelled validation subset.

Scope	Metric	Value
Primary-label	Accuracy	0.933
Primary-label	Macro Precision	0.950
Primary-label	Macro Recall	0.933
Primary-label	Macro F1	0.931
Multi-label	Exact Match Rate	0.267
Multi-label	Micro Precision	0.593
Multi-label	Micro Recall	0.941
Multi-label	Micro F1	0.727

Table D.2: Full MITRE mapping evaluation results against a 15-sample manually labelled validation subset.

Per-Label Evaluation Results

Table D.3 shows precision, recall, F1, and support for each technique in the primary-label evaluation.

Technique	Precision	Recall	F1	Support
T1566.001	0.750	1.000	0.857	3
T1566.002	1.000	1.000	1.000	4
T1598.001	1.000	0.667	0.800	3
T1598.002	1.000	1.000	1.000	2
T1598.003	1.000	1.000	1.000	3

Table D.3: Per-label precision, recall, and F1 for the primary-label MITRE mapping evaluation. Three of five techniques achieved perfect scores. **T1566.001** occasionally over-assigns (precision 0.75). **T1598.001** is the hardest to identify (recall 0.667).

Detection Rates by Technique in Simulation

Table D.4 shows how detection rates varied by MITRE technique across the full attacker simulation, providing a practical measure of model performance per technique category.

Technique	Total	TP	FN	Det.	Bypass	F1
T1566.002 — Phishing: Link	780	665	115	0.853	0.147	0.920
T1566.001 — Phishing: Attachment	135	95	40	0.704	0.296	0.826
T1566.001 — Attachment/Generic	45	5	40	0.111	0.889	0.200
N/A — Legitimate	60	0	0	0.000	0.000	0.000

Table D.4: Simulation detection rates by MITRE technique. The generic fallback category, assigned when no patterns match, has the lowest detection rate at 0.111 and the highest bypass rate at 0.889.

Attack Rule to MITRE Technique Mapping

Table D.5 shows which MITRE technique each attack rule maps to, and the detection rate achieved by that rule.

Rule	MITRE Technique	Det. Rate	Bypass
<code>fake_link</code>	T1566.002 — Phishing: Link	1.000	0.000
<code>urgency</code>	T1566.001 — Phishing: Attachment	0.833	0.167
<code>spoof_bank</code>	T1566.002 — Phishing: Link	0.583	0.417
<code>spoof_revenue</code>	T1566.001 — Attachment/Generic	0.167	0.833

Table D.5: Attack rule to MITRE technique mapping with single-rule detection rates. `spoof_revenue` maps to the generic fallback category because it contains no URL patterns or attachment keywords, producing the lowest detection rate in the simulation.

Bibliography

- [1] Michael Weinz, Cheng Zhu, Daniele Lain, Jacopo Cortellazzi, and Ben Stock. Assessing quishing and llm-generated phishing emails: An empirical analysis. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security (ASIACCS '25)*, New York, NY, USA, 2025. ACM.
- [2] Zhengbo Wang, Asiri Ratnayake, Tiansi Ma, Andrew Smith, Qing Chen, Wenjie Shen, Cheng Qian, and Yuhao Du. Phishemaillm: A comparative analysis of machine learning and large language models for phishing email detection. In *Proceedings of the Australasian Computer Science Week (ACSW '25)*, New York, NY, USA, 2025. ACM.
- [3] Junze Gu et al. Survey of explainable ai in phishing detection systems. In *2025 International Symposium on Networks, Computers and Communications (ISNCC)*, 2025.
- [4] Panagiotis Bountakas, Konstantinos Koutroumpouchos, and Christos Xenakis. A comparison of natural language processing and machine learning methods for phishing email detection. In *Proceedings of the 16th International Conference on Availability, Reliability and Security (ARES '21)*, 2021.
- [5] Nam V. Nguyen et al. Utilizing large language models with human feedback integration for phishing email detection. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIACCS '24)*, New York, NY, USA, 2024. ACM.
- [6] Ikenna Samuel Ibekwe, Fangyu Liu, and Weidong Fang. A systematic literature review on phishing email detection using natural language processing techniques. *IEEE Access*, 2022.
- [7] Aishwarya Gogoi and Md. Rafiqul Islam Ahmed. Phishing and fraudulent email detection using transfer learning. In *2022 IEEE India Council International Conference (INDICON)*, 2022.
- [8] Samy Namany, Moayad Al-Omari, and Mohammad Abu-Sbeih. Deep learning for phishing detection: Taxonomy, current challenges and future directions. *IEEE Access*, 2022.
- [9] Chris Otieno, Joshua Kagita, and Lawrence Nderu. Application of the bert transformer model for detecting phishing emails. In *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2023.

- [10] Ahmed Aljofey, Qian Jiang, Qiyan Qu, Xiaojie He, and Weimin Zhuang. Bert-phishfinder: An application of the bert transformer model for detecting phishing emails. *IEEE Transactions on Quantum Engineering*, 2025.
- [11] Raluca Dumitras, Angelina Davidescu, and Mihai Togan. Feature engineering and selection techniques for the detection of phishing emails. In *2024 International Conference on Communications (COMM)*, 2024.
- [12] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai. *Information Fusion*, 2020.
- [13] Pantelis Linardatos, Vasilis Papastefanopoulos, and Sotiris Kotsiantis. Explainable ai: A review of machine learning interpretability methods. *Entropy*, 2021.
- [14] Varun Verma, Anshika Raina, and Pritam Satam. Adversarial robustness of phishing email detection models. In *Proceedings of the 9th ACM International Workshop on Security and Privacy Analytics (IWSPA '23)*, New York, NY, USA, 2023. ACM.